

Support Copilot + MetricAI – Summary

A small support-ticket agent serving 3 simulated customers on 3 plans, with every LLM call routed through MetricAI so spend can be attributed by customer, ticket and workflow step.

Stack. Python 3.10, no agent framework — the workflow is a plain class with explicit steps. Metering runs through the `metricai` 0.8.4 SDK in BYOK mode; the smart tier calls Azure OpenAI (`gpt-4o`) through the OpenAI SDK and the cheap tier calls Gemini (`gemini-3.6-flash`) through `google-genai`. Every call is also written to a local JSONL ledger, so MetricAI's figures can be reconciled against my own.

Note. The tool call and the retrieval step are dummy setups: `check_outage_status` returns a random status without touching the network, and the knowledge base is eight hard-coded articles matched by keyword overlap. Both exist so the workflow has a non-LLM step and a token-inflating step to measure — neither is a real implementation.

1. Architecture

One ticket = one MetricAI **session**. Each step = its own `node_id`.

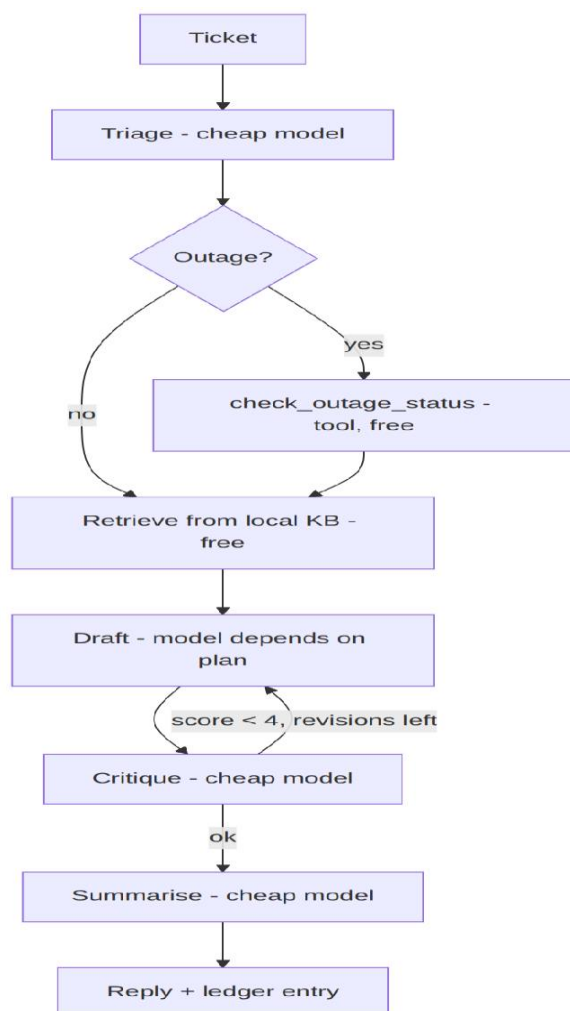


Figure 1 – Ticket workflow; each step carries its own `node_id`.

Plans differ in **cost to serve**, not just entitlements:

Plan	Price / ticket	Session cap	Draft model	Max revisions
Free	₹0	₹2	cheap	0
Pro	₹3	₹25	smart	1
Enterprise	₹9	₹200	smart	2

Every ticket looks up its tenant's plan before processing, and that plan decides how much reference material is retrieved, which model drafts the reply, and whether the draft is revised before it goes out:

- **Free** — the cheap model (Gemini) drafts the reply, knowledge-base enrichment is minimal, and the first draft ships as-is, with no revision loop.
- **Pro and Enterprise** — the smart model (Azure OpenAI) drafts the reply, enrichment goes deeper, and a critique/revision loop improves the draft before it is sent.

Every LLM call and tool call is logged with the plan and model tier used, so each ticket is fully auditable.

Outage handling is separate: it is triggered by the ticket category ("outage"), not by the tenant's plan, and behaves the same way on every plan.

2. Integration experience

I used **BYOK** mode — my own Gemini and Azure keys travel as headers to the MetricAI proxy, which forwards the call, meters it and attributes it. The vendor SDK calls stay the same; only `base_url` moves.

Attribution used: `agent_id`, `user_id` (customer), `session_id` (ticket), `graph_id` (workflow version), `node_id` (step) and `budget_cap_inr`.

Setup was quick, and model and provider spend showed up correctly in the portal.

3. Value observed

The useful part is not “what did we spend” but **where** it went: cost per ticket, per step and per customer, plus spend and forecast. None of that is visible on a cloud or LLM provider invoice.

Spend & Forecast

Track spend for the selected time range against budget pace and the projected month-end total.

[Export](#)[Adjust budget](#)

ACTUAL SPEND

₹2.662

Last 24 Hours

FORECASTED MONTH-END SPEND

₹92.24

Confidence range ₹27.67 – ₹156.81

BUDGET

No budget set

Create a budget to track pace and remaining spend

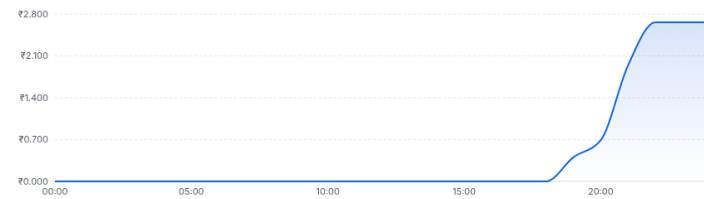
VARIANCE VS BUDGET

-

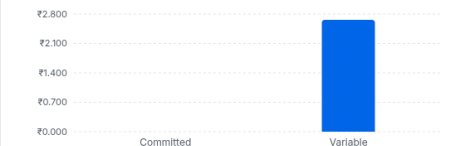
Set a budget to track variance

Last 24 Hours: 0.0289 USD - projecting 1.00 USD month-end at this burn.

Spend trend vs budget pace



Committed vs variable cost



Subscription commitments aren't tracked yet - spend shown here is usage-based.

Daily burn rate ₹2.574/day

Days remaining in period 12

Spend drivers (vs previous period)

support-copilot

₹0.000 → ₹3.106

+100.0%

Filters

Project

All projects

Figure 2 — Spend and forecast for the agent.

Cost Allocation

See exactly where spend is going across every dimension that matters financially.

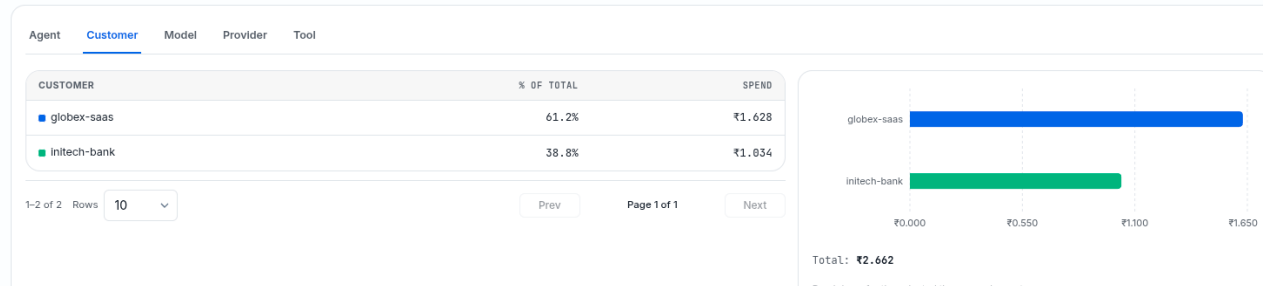


Figure 3 — Cost allocation by customer.

Cost Allocation

See exactly where spend is going across every dimension that matters financially.

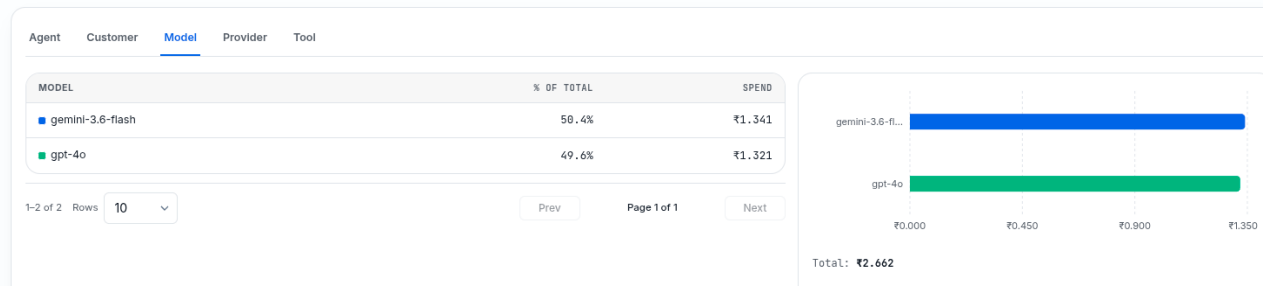


Figure 4 — Cost allocation by model.

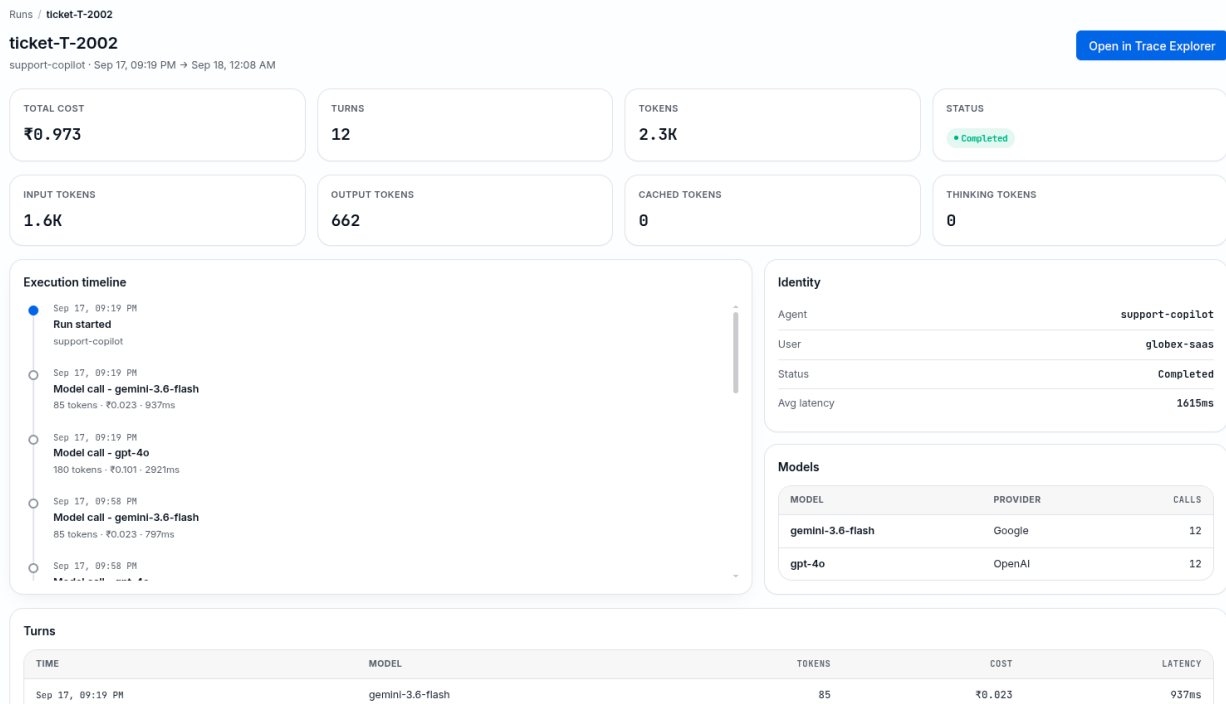


Figure 5 — A single ticket, broken down call by call.

4. Problems encountered

What I could not work out: tool calls. I report the non-LLM step through `mc.track(...)` as `tool:check_outage_status`, but the **Tools tab in the portal stays empty** — the 8 requests instead land under a model named `Unknown` with provider `Other` (below). I could not find anything in the documentation explaining the expected payload for a tool event, so I do not know whether my call is wrong or the tab is not wired up.

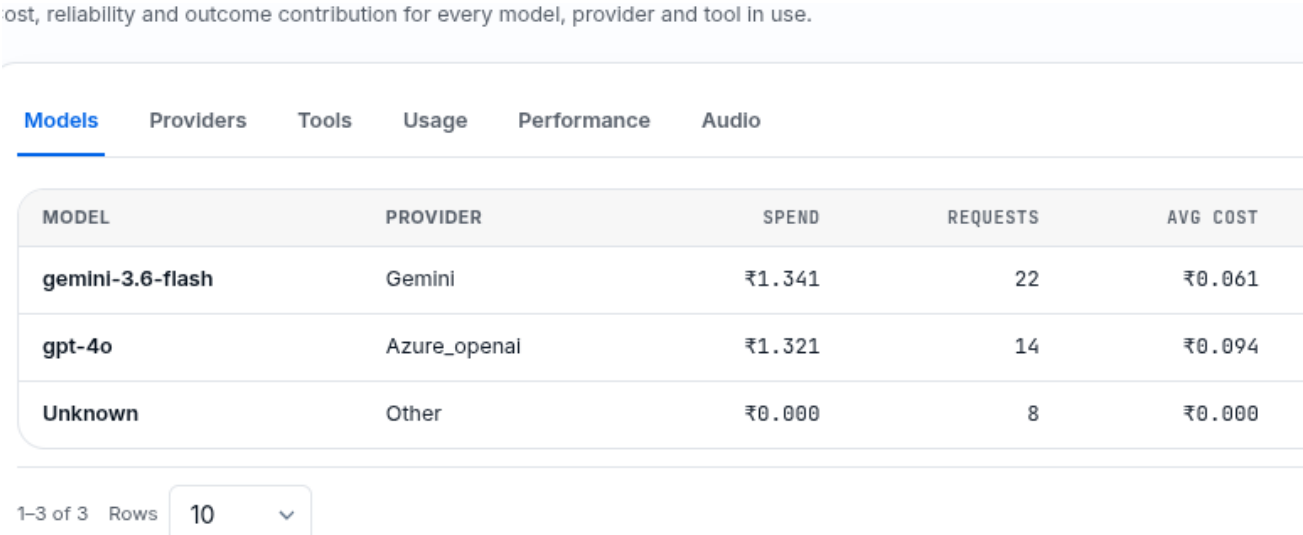


Figure 6 — The 8 tool events counted as model Unknown, provider Other.

ost, reliability and outcome contribution for every model, provider and tool in use.

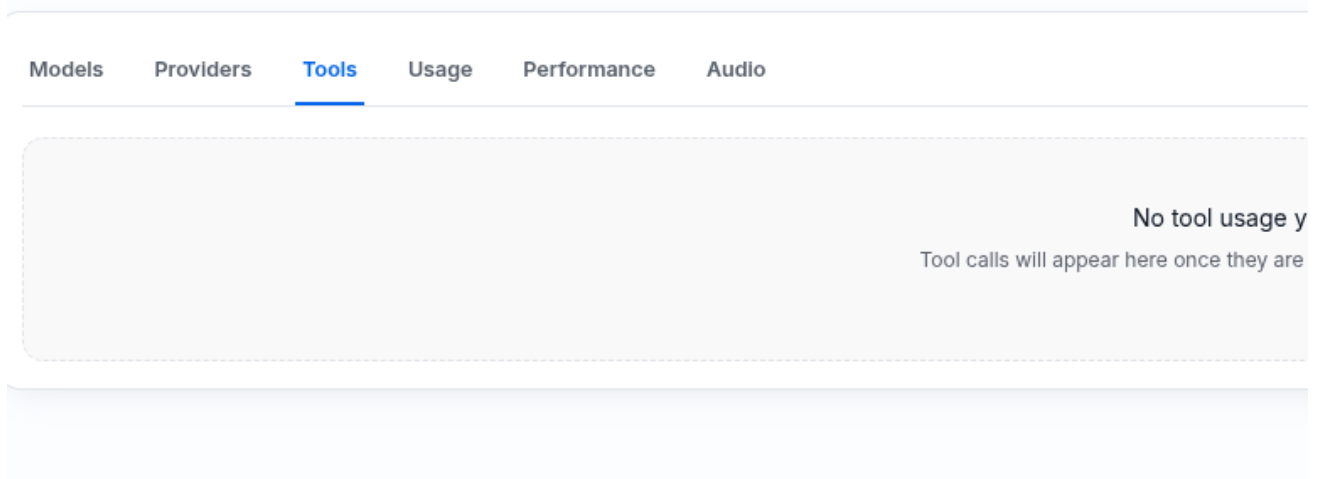


Figure 7 – The Tools tab stays empty.

5. Critical feedback

- **Tool-call tracking is not documented anywhere** – the only mention is a single line saying that events get sent, with nothing on the fields or values expected.
- **Azure is listed as a supported provider**, but no page shows the setup details: which keys to use, and deployment name vs. model name.

6. Product improvement suggestions

- **JS/Node SDK gap.** `metricai.co.in`'s landing page claims "Python, Node.js" support, but there is no `metricai` package on npm, the docs' install instructions are Python-only, and the installed PyPI package describes itself as "Python SDK for MetricAI." That is both a market-expansion opportunity – JS/TS is where most agent frameworks deploy – and a trust risk, since a false claim on the pricing page casts doubt on other claims, such as in-path budget enforcement.
- **REST API as a first-class documented path.** Teams on Go, Ruby or Java are otherwise blocked waiting on SDK releases.
- **Custom dashboards.** Let me build my own view: drag existing panels onto a canvas, and describe a chart that does not exist yet in a prompt, so it is built from the events already stored.